Content

이번 주차의 학습목표와 학습내용을 확인하세요.

01. 정렬의 개념과 특징

정렬의 개념과 간단한 정렬 알고리즘의 동작 원리를 설명할 수 있다.

02. 기본 정렬

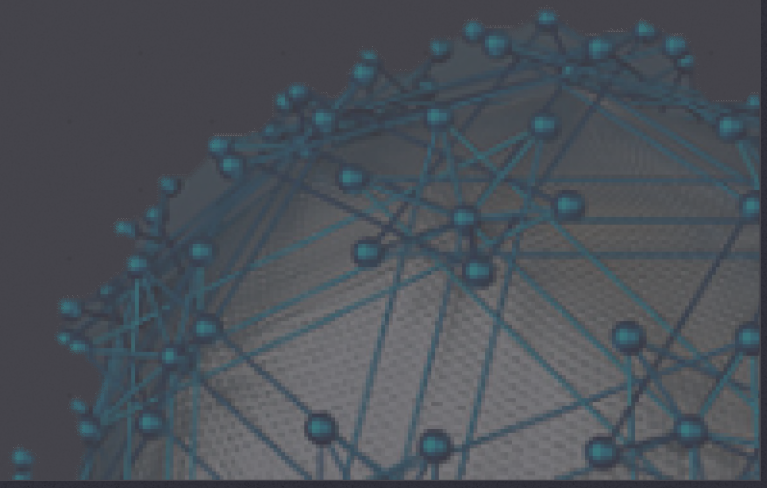
선택/삽입/버블 등 기본 정렬과 집합 관련 연산의 효율을 향상시키는 방법에 대해 설명할 수 있다.

정렬의 개념과 특징

정렬의 개념과 특징

정렬

- 데이터를 순서대로 재배열한 것
- 가장 기본적이고, 중요한 알고리즘

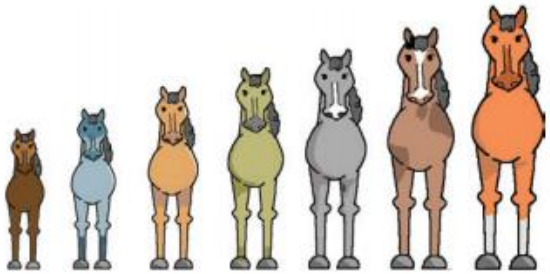


정렬의 개념과 특징

정렬의 개념

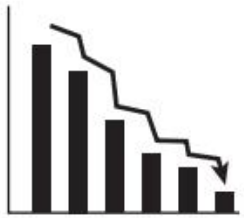
비교할 수 있는 모든 속성들은 정렬의 기준이 될 수 있음

예 오름차순 정렬, 내림차순 정렬, 키순



경주마의 정렬(키 순)

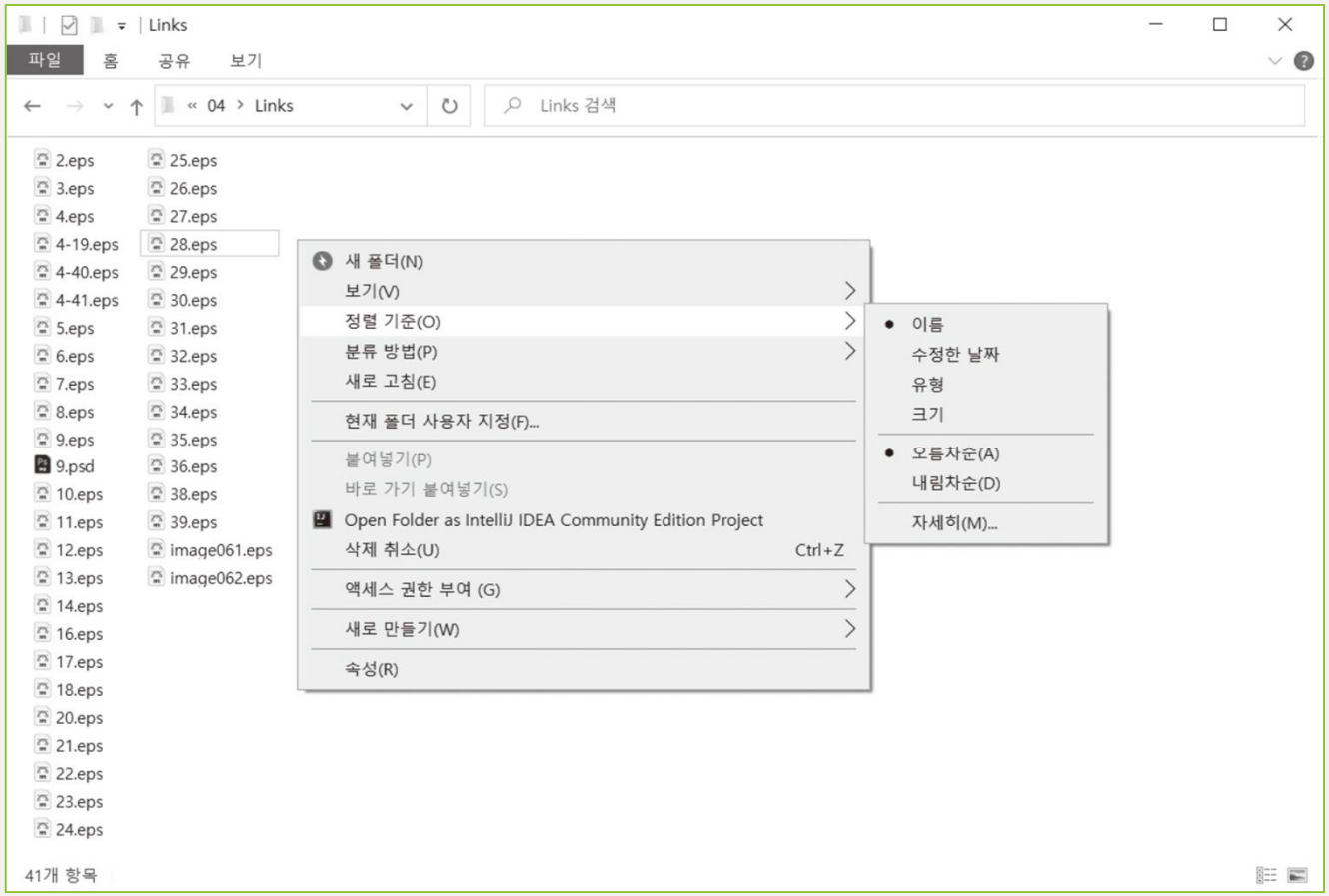
오름차순 정렬



내림차순 정렬

정렬의 개념과 특징

생활 속의 정렬



정렬의 개념과 특징

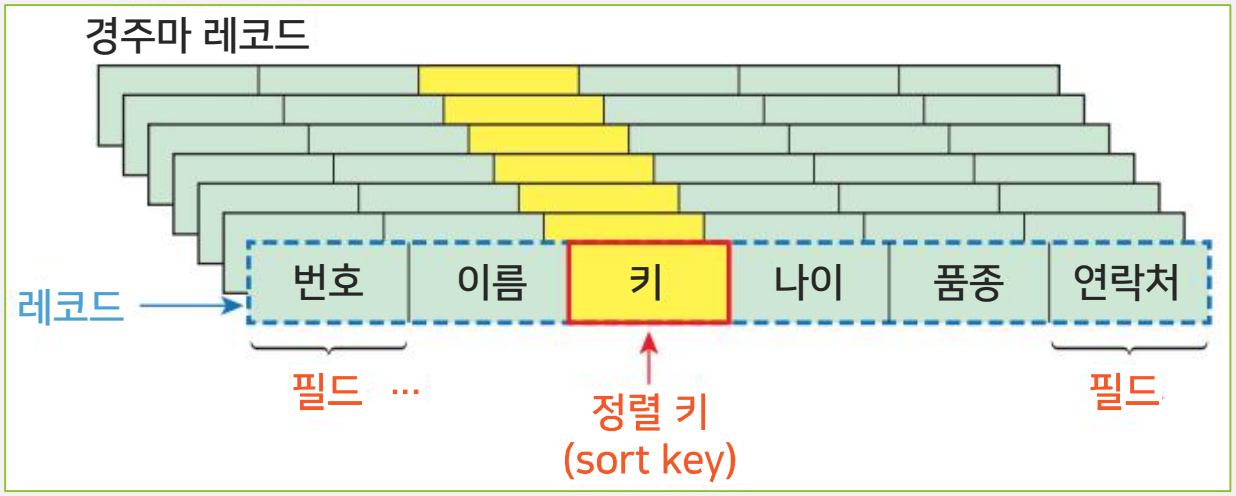
용어들

레코드

정렬시켜야 될 대상

여러 개의 필드(field)로 이루어짐

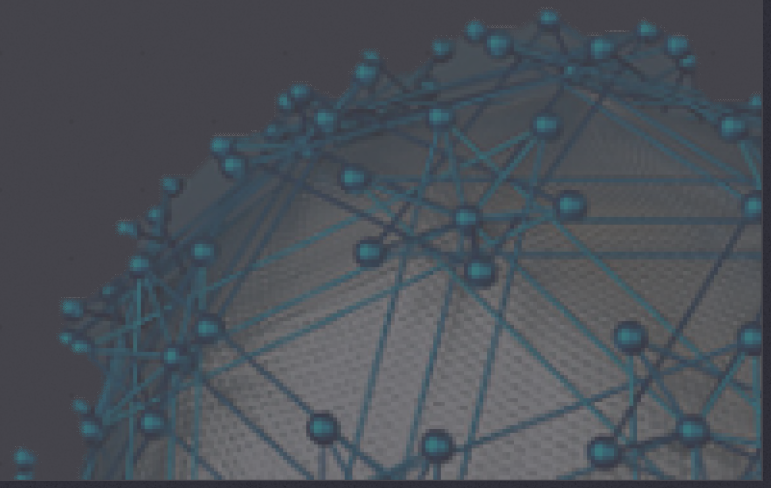
필드들 중 정렬의 기준이 되는 필드인 정렬 키(sort key)가 있음



정렬의 개념과 특징

정렬

레코드들을 키(key)의 순서로
재배열하는 것



정렬의 개념과 특징

효율성에 따른 분류

단순하지만 비효율적인 방법

- 삽입 정렬, 선택 정렬, 버블 정렬 등

선택 정렬
버블 정렬
삽입 정렬

→ $\theta(n^2)$

복잡하지만 효율적인 방법

- 퀵 정렬, 병합 정렬, 셀 정렬 등

병합 정렬
퀵 정렬
힙 정렬
셀힙 정렬

→ $\theta(n\log n)$

데이터 특성을 잘 이용하는 방법

- 계수 정렬, 기수 정렬, 버킷 정렬 등

계수 정렬
기수 정렬
버킷 정렬

→ $\theta(n)$

정렬의 개념과 특징

정렬 장소에 따른 분류

내부 정렬	외부 정렬
▶ 모든 데이터가 주기억장치에 있음 ▶ 모든 데이터를 주기억장치에 저장한 후 정렬하는 방식	▶ 외부 기억장치에 대부분의 레코드가 있음 ▶ 모든 데이터를 주기억장치에 저장할 수 없는 경우, 일부 데이터만 주기억장치에 있고 나머지는 외부기억장치에 저장한 채 정렬하는 방식

정렬의 개념과 특징

키 값의 비교 여부에 따른 분류

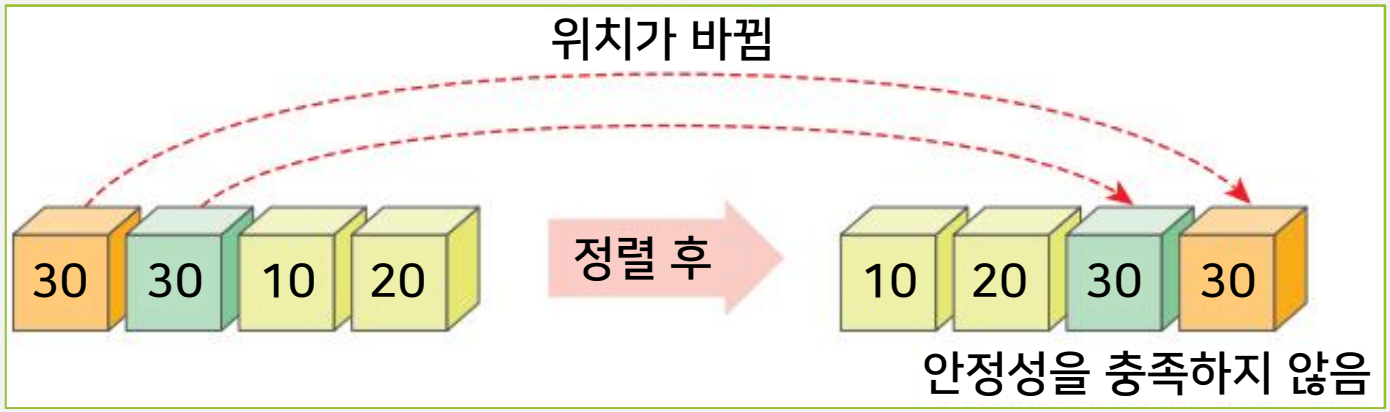
키값의 "비교"를 이용한 정렬	"배분"을 이용한 정렬
▶ 대부분의 정렬이 포함	▶ 기수 정렬, 카운팅 정렬이 포함

정렬의 개념과 특징

정렬 알고리즘의 특징

안정성(Stability)

동일한 키값의 레코드가 정렬 후 상대적인 위치를 유지하는가?



➡ 정렬 후에도 본래의 위치를 유지한 경우, 안정성을 충족

정렬의 개념과 특징

정렬 알고리즘의 특징

제자리 정렬

추가적인 메모리 공간을 필요하는가?

입력의 종류에 따른 성능 차이

입력의 종류와 상관 없이 동일한 시간이 걸리는가?

입력의 종류에 따라 빠르게 혹은 느리게 동작하는 알고리즘이 있는
반면 입력의 종류와 상관 없이 동일한 시간이 걸리는 알고리즘이 있음

기본 정렬

선택 정렬과 삽입 정렬, 버블 정렬

선택 정렬의 아이디어

선택 정렬

▶ 오른쪽 리스트에서 가장 작은 숫자를 선택해 왼쪽 리스트의 맨 뒤로 이동하는 작업을 반복

정렬된(왼쪽) 리스트	정렬 안 된(오른쪽) 리스트	설명
[]	[5,3,8,4,9,1,6,2,7]	초기상태
[1]	[5,3,8,4,9,6,2,7]	1선택 및 이동
[1,2]	[5,3,8,4,9,6,7]	2선택 및 이동
[1,2,3]	[5,8,4,9,6,7]	3선택 및 이동
…	…	4~8선택 및 이동
[1,2,3,4,5,6,7,8,9]	[]	9선택 및 이동

➡ 1번 작업할 때마다, 오른쪽 리스트의 크기는 1씩 감소함에 따라 왼쪽 리스트는 1씩 증가

선택 정렬과 삽입 정렬, 버블 정렬

선택 정렬의 전략

제자리 정렬 전략

- 입력 리스트를 두 부분으로 나누어 사용

  정렬된 부분과 정렬 안된 부분

- 정렬 안된 부분에서 가장 작은 숫자를 찾아 첫 번째 원소와 교환

  정렬된 부분이 점점 커지고, 정렬 안된 부분이 줄어듦

➡ 정렬되지 않은 부분에서 최소값을 찾아 정렬되지 않은 리스트의 가장 첫 항목과 교환

선택 정렬과 삽입 정렬, 버블 정렬

선택 정렬 과정 예

단위 순환

- 최소값을 찾는다.
- 최소값과 맨 왼쪽 원소를 자리 바꾼다.
- 맨 왼쪽 자리를 관심 대상에서 제외한다.

➡ 원소가 1개 남을 때까지 위의 순환을 반복

선택 정렬과 삽입 정렬, 버블 정렬

선택 정렬 알고리즘

```
def selection_sort( A ) :
    n = len(A)
    for  i  in  range(n-1)  :
        least = i
        for j in range(i+1, n) :
            if (A[j] < A[least]) :
                least = j
        A[i], A[least] = A[least], A[i]    # 배열 항목 교환
        printStep (A, i+1)                 # 중간 과정 출력용 문장
```

선택 정렬과 삽입 정렬, 버블 정렬

선택 정렬 테스트 프로그램

```
data = [5, 3, 8, 4, 9, 1, 6, 2, 7]
print("Original : ", data)
selection_sort(data)
print("Selection : ", data)
```

C:₩WINDOWS₩system32₩cmd.exe

```
Original  : [5, 3, 8, 4, 9, 1, 6, 2, 7]
   Step 1 = [1, 3, 8, 4, 9, 5, 6, 2, 7]
   Step 2 = [1, 2, 8, 4, 9, 5, 6, 3, 7]
   Step 3 = [1, 2, 3, 4, 9, 5, 6, 8, 7]
   Step 4 = [1, 2, 3, 4, 9, 5, 6, 8, 7]
   Step 5 = [1, 2, 3, 4, 5, 9, 6, 8, 7]
   Step 6 = [1, 2, 3, 4, 5, 6, 9, 8, 7]
   Step 7 = [1, 2, 3, 4, 5, 6, 7, 8, 9]
   Step 8 = [1, 2, 3, 4, 5, 6, 7, 8, 9]
Selection : [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

정렬 안 된 리스트

정렬된 리스트

선택 정렬과 삽입 정렬, 버블 정렬

선택 정렬의 시간 복잡도

시간 복잡도

$(n-1) + (n+2) + \cdots + 1 = n(n-1)/2 = O(n^2)$

리스트의 크기가 정해지면 자료 이동 횟수가 미리 결정

최선, 최악의 복잡도 동일

선택 정렬과 삽입 정렬, 버블 정렬

선택 정렬의 특징

선택 정렬의 장점	선택 정렬의 단점
▶ 알고리즘이 간단 ▶ 제자리 정렬 ▶ 자료 이동 횟수가 미리 결정	▶ 속도가 느림 ▶ 안정성을 만족하지 않음 예) [3,2,3,1] [1,2,3,3] → [1,2,3,3] → [1,2,3,3] → [1,2,3,3]

선택 정렬과 삽입 정렬, 버블 정렬

삽입 정렬의 아이디어

삽입 정렬

- Insertion sort
- 정렬되어 있는 부분에 새로운 레코드를 올바른 위치에 삽입하는 과정 반복

선택 정렬과 삽입 정렬, 버블 정렬

삽입 정렬의 전략

제자리 정렬 전략

- 입력 리스트를 두 부분으로 나누어 사용

  정렬된 부분과 정렬 안된 부분

- 오른쪽 리스트의 첫 항목을 순서대로 왼쪽에 끼워 넣음

끼워 넣기

- 많은 항목들의 이동이 발생
- 끼워 넣을 위치에 따라 이동할 항목의 수가 다름

  최악의 경우? 최선의 경우?

➡ 끼워 넣은 위치가 가장 마지막이라면, 다른 항목의 이동 없이 마지막에 추가할 수 있음
입력 데이터에 따라 시간이 많거나 적게 걸릴 수 있음

선택 정렬과 삽입 정렬, 버블 정렬

삽입 정렬 과정 예

선택 정렬과 삽입 정렬, 버블 정렬

삽입 정렬 알고리즘

```
def insertion_sort( A ) :
    n = len(A)
    for  i  in  range(1, n)  :              # 두 번째 항목부터 모든 항목을
        key = A[i]                          # A[i] = 정렬되지 않은 리스트의 첫 항목
        j = i -1
        while j >= and A[j] > key :  # 이후 항목들의 이동이 필요
            A[j+1] < A[j])
            j -= 1
        A[j+1] = key
        printStep (A, i)
```

선택 정렬과 삽입 정렬, 버블 정렬

삽입 정렬 테스트 프로그램

```
data = [5, 3, 8, 4, 9, 1, 6, 2, 7]
print("Original : ", data)
insertion_sort(data)
print("Insertion : ", data)
```

C:₩WINDOWS₩system32₩cmd.exe

```
Original  : [5, 3, 8, 4, 9, 1, 6, 2, 7]
   Step 1 = [3, 5, 8, 4, 9, 1, 6, 2, 7]
   Step 2 = [3, 5, 8, 4, 9, 1, 6, 2, 7]
   Step 3 = [3, 4, 5, 8, 9, 1, 6, 2, 7]
   Step 4 = [3, 4, 5, 8, 9, 1, 6, 2, 7]
   Step 5 = [1, 3, 4, 5, 8, 9, 6, 2, 7]
   Step 6 = [1, 3, 4, 5, 6, 8, 9, 2, 7]
   Step 7 = [1, 2, 3, 4, 5, 6, 8, 9, 7]
   Step 8 = [1, 2, 3, 4, 5, 6, 7, 8, 9]
Insertion : [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

정렬 안 된 부분

정렬된 부분

선택 정렬과 삽입 정렬, 버블 정렬

삽입 정렬의 시간 복잡도

최선의 경우 O(n)	이미 정렬되어 있는 경우(비교 n-1번)
최악의 경우 $O(n^2)$	역순으로 정렬되어 있는 경우

- 역순으로 정렬되어 있어 모든 단계에서 앞에 놓인 자료 전부 이동
- 비교: $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = O(n^2)$
- 이동: $\frac{n(n-1)}{2} + 2(n-1) = O(n^2)$

평균 경우	$O(n^2)$

선택 정렬과 삽입 정렬, 버블 정렬

삽입 정렬의 특징

삽입 정렬의 장점	삽입 정렬의 단점
▶ 알고리즘이 간단 ▶ 제자리 정렬 ▶ 안정성을 만족	▶ 속도가 느림 ▶ 레코드가 큰 경우, 많은 이동이 필요 ▶ 대부분 정렬되어 있다면 효율적

선택 정렬과 삽입 정렬, 버블 정렬

버블 정렬의 아이디어

버블 정렬(bubble sort)

인접한 2개의 레코드를 비교해 순서대로 서로 교환

비교-교환 과정을 리스트 전체에 수행(스캔)

1번의 스캔이 완료되면 리스트의 오른쪽 끝에 가장 큰 레코드가 들어감

선택 정렬과 삽입 정렬, 버블 정렬

버블 정렬 과정 예

5	3	8	4	9	1	6	2	7	<스캔 1> 5와 3 교환
3	5	8	4	9	1	6	2	7	5와 8은 제자리
3	5	8	4	9	1	6	2	7	8과 4 교환 동일 작업 반복
⋮									
3	5	4	8	1	6	2	7	9	<스캔 1 완료>
3	4	5	1	6	2	7	8	9	<스캔 2 완료>
3	4	1	5	2	6	7	8	9	<스캔 3 완료>
⋮									
1	2	3	4	5	6	7	8	9	<스캔 6 완료>

단위 순환

- 맨 앞부터 한 칸씩 이동하면서 두 원소가 순서대로 되어 있지 않으면 자리 바꾼다.
- 맨 오른쪽 자리를 관심 대상에 제외한다.

➡ 원소가 1개 남을 때까지 위의 순환을 반복

➡ 교환이 발생하지 않는다면, 더 이상 스캔할 필요가 없음

선택 정렬과 삽입 정렬, 버블 정렬

버블 정렬 알고리즘

```
def bubble_sort( A ) :
    n = len(A)
    for  i  in  range(n-1, 0, -1)  :
        bChanged = False = A[i]
        for j in range(i) :
            if ( A[j] > A[j+1]) :
                A[j] < A[j+1] = A[j+1], A[j]
                bChanged = True
        if not bChanged :   break          # 교환이 없으면 종료
        printStep (A, n-i)                 # 중간 과정 출력용 문장
```

선택 정렬과 삽입 정렬, 버블 정렬

버블 정렬 테스트 프로그램

```
data = [5, 3, 8, 4, 9, 1, 6, 2, 7]
print("Original : ", data)
bubble_sort(data)
print("Bubble : ", data)
```

C:₩WINDOWS₩system32₩cmd.exe

```
Original  : [5, 3, 8, 4, 9, 1, 6, 2, 7]
  Step 1 = [3, 5, 4, 8, 1, 6, 2, 7, 9]
  Step 2 = [3, 4, 5, 1, 6, 2, 7, 8, 9]
  Step 3 = [3, 4, 1, 5, 2, 3, 7, 8, 9]
  Step 4 = [3, 1, 4, 2, 5, 6, 7, 8, 9]
  Step 5 = [1, 3, 2, 4, 5, 6, 7, 8, 9]
  Step 6 = [1, 2, 3, 4, 5, 6, 7, 8, 9]
Bubble    : [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

항목을 교환할 범위

정렬된 부분

더 이상 교환이 일어나지 않음. 정렬 종료

선택 정렬과 삽입 정렬, 버블 정렬

버블 정렬의 시간 복잡도

비교 횟수	이동 횟수
$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = O(n^2)$$	▶ 역순으로 연결된 경우(최악) : 이동 횟수 = 3* 비교 횟수 ▶ 이미 정렬된 경우(최선) : 이동 횟수 = 0 ▶ 평균의 경우 : $O(n^2)$

비교 횟수(최상, 평균, 최악의 경우 모두 일정)

➡ 바깥 루프는 n-1번 반복/바깥 루프가 1번 반복할 때마다, 내부 루프는 i 만큼 반복

선택 정렬과 삽입 정렬, 버블 정렬

버블 정렬의 특징

버블 정렬의 장점	버블 정렬의 단점
▶ 알고리즘이 간단 ▶ 제자리 정렬 ▶ 안정성을 만족	▶ 최악 혹은 평균적인 경우의 시간 복잡도가 $O(n^2)$로, 속도가 느림 ▶ 레코드의 이동이 과다하지만, 최선의 경우와 같이 어느 정도 정렬되어 있다면 효과적

학습평가

번호	문제	보기
1	다음 중 단순하지만 비효율적인 정렬이 아닌 것은?	① 퀵 정렬 ② 삽입 정렬 ③ 버블 정렬 ④ 선택 정렬
2	다음 보기를 읽고 맞으면 O, 틀리면 X를 선택하세요.	내부 정렬은 모든 데이터를 주기억장치에 저장할 수 없는 경우, 일부 데이터만 주기억장치에 있고 나머지는 외부기억장치에 저장한 채 정렬하는 방식 이다. ① O ② X
3	다음 중 가장 옳지 않은 것은?	① 선택 정렬은 오른쪽 리스트에서 가장 작은 숫자를 선택해 왼쪽 리스트의 맨 뒤로 이동하는 작업을 반복한다. ② 선택 정렬은 정렬 안된 부분에서 가장 작은 숫자를 찾아 첫 번째 원소와 교환한다. ③ 삽입 정렬은 정렬되어 있지 않은 부분에 새로운 레코드를 올바른 위치에 삽입하는 과정을 반복한다. ④ 삽입 정렬은 입력 리스트를 두 부분으로 나누어 사용한다.
4	다음 중 가장 옳지 않은 것은?	① 삽입 정렬은 오른쪽 리스트의 첫 항목을 순서대로 왼쪽에 끼워 넣는다. ② 버블 정렬은 인접한 2개의 레코드를 비교해 순서대로 서로 교환한다. ③ 버블 정렬은 비교-교환 과정을 리스트를 둘로 나누어 왼쪽 부분만 수행한다. ④ 버블 정렬에서 1번의 스캔이 완료되면 리스트의 오른쪽 끝에 가장 큰 레코드가 들어간다.

학습정리

정렬의 개념과 특징

정렬의 개념

- 용량이 데이터를 순서대로 재배열한 것
- 비교할 수 있는 모든 속성들은 정렬의 기준이 될 수 있음

정렬 알고리즘의 효율성에 따른 분류

- 단순하지만 비효율적인 방법 : 삽입정렬, 선택 정렬, 버블 정렬 등
- 복잡하지만 효율적인 방법 : 퀵 정렬, 병합 정렬, 셸 정렬 등
- 데이터 특성을 잘 이용하는 방법 : 계수 정렬, 기수 정렬, 버킷 정렬 등

정렬 장소에 따른 분류

- 내부 정렬 : 모든 데이터가 주기억장치에 있음
- 외부 정렬 : 외부 기억장치에 대부분의 레코드가 있음

키 값의 비교 여부에 따른 분류

- 키값의 "비교"를 이용한 정렬 : 대부분의 정렬
- 배분을 이용한 정렬 : 기수 정렬, 카운팅 정렬 등

학습정리

기본 정렬

선택 정렬

- 오른쪽 리스트에서 가장 작은 숫자를 선택해 왼쪽 리스트의 맨 뒤로 이동하는 작업을 반복
- 입력 리스트를 두 부분으로 나누어 사용
- 정렬 안된 부분에서 가장 작은 숫자를 찾아 첫 번째 원소와 교환

삽입 정렬

- 정렬되어 있는 부분에 새로운 레코드를 올바른 위치에 삽입하는 과정 반복
- 입력 리스트를 두 부분으로 나누어 사용
- 오른쪽 리스트의 첫 항목을 순서대로 왼쪽에 끼워 넣음

버블 정렬

- 인접한 2개의 레코드를 비교해 순서대로 서로 교환
- 비교-교환 과정을 리스트 전체에 수행(스캔)
- 1번의 스캔이 완료되면 리스트의 오른쪽 끝에 가장 큰 레코드가 들어감